



NGUYÊN LÝ HỆ ĐIỀU HÀNH

Phần 5: Đồng bộ hoá tiến trình

NGUYỄN THỊ HẬU

Khoa Công Nghệ Thông Tin
Đại học Công Nghệ - Đại học quốc gia Hà Nội

Phần 5: Đồng bộ hoá tiến trình

1. Bài toán khu vực quan trọng
2. Giải pháp Peterson
3. Đồng bộ hoá phần cứng
4. Semaphore
5. Một số bài toán đồng bộ hoá cơ bản
6. Một số vấn đề với Semaphore

Phần 5: Đồng bộ hoá tiến trình

5.1 Bài toán khu vực quan trọng

Ví dụ

- Truy cập đồng thời vào phần dữ liệu chia sẻ có thể dẫn đến tình trạng dữ liệu không nhất quán
- Ví dụ:

Nhà sản xuất (P)

```
while (true) { /* produce an item and put in
nextProduced */
while (counter == BUFFER_SIZE); // do
nothing
buffer [in] = nextProduced;
in = (in + 1) % BUFFER_SIZE;
counter++;
}
```

Người dùng (C)

```
while (true) {
while (counter == 0); // do nothing
nextConsumed = buffer[out];
out = (out + 1) % BUFFER_SIZE;
counter--;
/* consume the item in nextConsumed */
}
```

Ví dụ

- Các toán tử ++ và -- có thể được cài đặt như sau:

- counter++

```
register1 = counter;
```

```
register1 = register1 + 1;
```

```
counter = register1;
```

- counter--

```
register2 = counter;
```

```
register2 = register2 - 1;
```

```
counter = register2;
```

P và C có thể nhận được các giá trị khác nhau của counter tại cùng 1 thời điểm nếu như đoạn mã **xanh** và **đỏ** thực hiện xen kẽ nhau.

- Tình huống xuất hiện khi nhiều tiến trình cùng thao tác trên dữ liệu chung và kết quả các thao tác đó phụ thuộc vào thứ tự thực hiện của các tiến trình trên dữ liệu chung gọi là *tình huống tương tranh (race condition)*
- Để tránh các tình huống tương tranh, các tiến trình cần được *đồng bộ* theo một phương thức nào đó $\Rightarrow$ Vấn đề nghiên cứu: Đồng bộ hóa các tiến trình

Bài toán khu vực quan trọng

- Có n tiến trình trong hệ thống $\{p_0, p_1, \dots, p_{n-1}\}$
- Mỗi tiến trình có một đoạn mã truy cập vào khu vực quan trọng (**critical section - CS**)
 - Để thay đổi giá trị các biến toàn cục, cập nhật, viết tệp,...
 - Khi một tiến trình thực thi khu vực quan trọng, không một tiến trình nào khác được thực thi khu vực quan trọng
- Mỗi tiến trình phải yêu cầu quyền truy cập vào khu vực quan trọng

Bài toán khu vực quan trọng

- Cấu trúc chung của một tiến trình khi sử dụng khu vực quan trọng:

do {

entry section

critical section

exit section

remainder section

} while (true);

Các điều kiện cần đảm bảo để giải quyết bài toán khu vực quan trọng

9

- **Loại trừ lẫn nhau (Mutual Exclusion):** Khi một tiến trình P_i đang thực thi khu vực quan trọng (CS_i), không một tiến trình nào khác được thực thi khu vực quan trọng
- **Tiến triển (Progress):** Nếu không có tiến trình nào đang thực thi khu vực quan trọng, và tồn tại một số tiến trình muốn thực thi khu vực quan trọng thì việc lựa chọn tiến trình tiếp theo thực thi khu vực quan trọng không thể bị trì hoãn vô thời hạn
- **Chờ đợi có giới hạn (Bounded Waiting):** *sau khi* một tiến trình P_i có yêu cầu vào CS_i và *trước khi* yêu cầu đó được chấp nhận, số lần các tiến trình P_j (với $j \neq i$) được phép thực hiện CS_j phải bị giới hạn

Phần 5: Đồng bộ hoá tiến trình

5.2 Giải pháp Peterson

Giải pháp Peterson

- *Hai tiến trình* chia sẻ hai tham số:
 - int **turn**;
 - Boolean **flag[2]**
- Tham số **turn** chỉ tiến trình đến lượt vào CS
- **flag[i] = true** chỉ tiến trình P_i đã sẵn sàng vào CS

Giải pháp Peterson

■ Cấu trúc của tiến trình P_i

do {
 flag[i] = TRUE;
 turn = j;
 while (flag[j] && turn == j); //wait to enter CS
 critical section
 flag[i] = FALSE;
 remainder section
} while (TRUE);

■ Hãy chứng minh ba tính chất Loại trừ lẫn nhau, Tiến triển, Chờ đợi có giới hạn được đảm bảo

Phần 5: Đồng bộ hoá tiến trình

5.3 Đồng bộ hoá phần cứng

Đồng bộ hoá phần cứng

- Giải pháp cho vấn đề khu vực quan trọng thông qua **Khoá**:
do {
 acquire lock
 critical section
 release lock
 remainder section
} while (TRUE);

Đồng bộ hoá phần cứng trong hệ đơn vi xử lý

15

- Vô hiệu hoá hoạt động của các ngắt khi đang cập nhật các biến chung

- VD:

```
boolean test and set(boolean *target) {  
    boolean rv = *target;  
    *target = true;  
    return rv;  
}
```

Đồng bộ hoá phần cứng trong hệ đơn vi xử lý

16

- Đoạn mã đảm bảo tính loại trừ lẫn nhau

do {

while (test and set(&lock)); /* do nothing */

/* critical section */

lock = false;

/* remainder section */

}

Đồng bộ hoá phần cứng trong hệ đơn vi xử lý

17

- Đoạn mã đảm bảo tính loại trừ lẫn nhau + Chờ đợi có giới hạn

```
do {                                     if (j == i)
waiting[i] = TRUE;                       lock = FALSE;
key = TRUE;                             else
while (waiting[i] && key)                 waiting[j] = FALSE;
    key = TestAndSet(&lock); // remainder section
waiting[i] = FALSE;                     } while (TRUE);
// critical section
j = (i + 1) % n;
while ((j != i) && !waiting[j])
    j = (j + 1) % n;
```

Đồng bộ hoá phần cứng trong hệ đa vi xử lý

18

- Sử dụng lệnh phần cứng không cho phép ngắt (atomic hardware instructions)

- VD:

```
void Swap (boolean *a, boolean *b)
{
    boolean temp = *a;
    *a = *b;
    *b = temp;
}
```

Đồng bộ hoá phần cứng trong hệ đa vi xử lý

- Giải pháp cho vấn đề khu vực quan trọng:
- Biến chung **Lock** được khởi tạo bằng FALSE, mỗi tiến trình có biến địa phương **key**

```
do {  
    key = TRUE;  
    while ( key == TRUE)  
        Swap (&lock, &key );  
        // critical section  
    lock = FALSE;  
        // remainder section  
} while (TRUE);
```

- *Đoạn mã lệnh trên đảm bảo những tính chất nào ? (Loại trừ lẫn nhau, Tiến triển, Chờ đợi có giới hạn)*

Phần 5: Đồng bộ hoá tiến trình

5.4 Semaphore

Khái niệm Semaphore

- Semaphore S – biến số ~~tự nhiên~~ nguyên
- Hai hàm chuẩn thay đổi giá trị của S: `wait()` và `signal()`

```
wait (S) {  
    while S <= 0; // busy  
wait  
    S--;  
}
```

```
signal (S) {  
    S++;  
}
```

- Hai hàm nguyên tử
- Tại mỗi thời điểm, nhiều nhất một tiến trình được thay đổi giá trị của S

Các kiểu Semaphore

- Semaphore đếm: giá trị biến tự nhiên trong một miền giới hạn
- Semaphore nhị phân: 0 hoặc 1
 - còn gọi là Khoá mutex
 - Đảm bảo tính loại trừ lẫn nhau:

Semaphore mutex; // Khởi tạo bằng 1

```
do {  
    wait (mutex);  
    // Critical Section  
    signal (mutex);  
    // remainder section  
} while (TRUE);
```

Triển khai Semaphore nhằm tránh tình trạng bận rộn chờ đợi

23

- Mỗi semaphore có một hàng đợi
- Mỗi lượt vào hàng đợi có 2 dữ liệu
 - Giá trị tự nhiên
 - Con trỏ đến bản ghi tiếp theo trong danh sách
- Hai hàm
 - **block()**: đưa tiến trình gọi hàm này vào hàng đợi thích hợp
 - **wakeup()**: loại bỏ một trong các tiến trình ở hàng đợi và đưa nó vào hàng đợi sẵn sàng

Triển khai Semaphore nhằm tránh tình trạng bận rộn chờ đợi

24

```
wait(semaphore *S) {  
    S->value--;  
  
    if (S->value < 0) {  
        add this process to S->list;  
        block();  
    }  
}
```

```
signal(semaphore *S) {  
    S->value++;  
  
    if (S->value <= 0) {  
        remove a process P  
        from S->list;  
        wakeup(P);  
    }  
}
```


Phần 5: Đồng bộ hoá tiến trình

5.5 Một số bài toán đồng bộ hoá cơ bản

Bài toán vùng đệm có giới hạn

26

- N vùng đệm, mỗi vùng đệm chứa được một dữ liệu
- Khởi tạo semaphore **mutex** bằng 1 // Đảm bảo tính loại trừ lẫn nhau
- Khởi tạo semaphore **full** bằng 0 // Đếm số vùng đệm chứa dữ liệu
- Khởi tạo semaphore **empty** bằng N // Đếm số vùng đệm trống

Bài toán vùng đệm có giới hạn

■ Nhà sản xuất:

```
do {  
  
    // produce an item in nextp  
  
    wait (empty);  
  
    wait (mutex);  
  
    // add the item to the buffer  
  
    signal (mutex);  
  
    signal (full);  
  
} while (TRUE);
```

Bài toán vùng đệm có giới hạn

■ Người dùng

```
do {  
    wait (full);  
    wait (mutex);  
    // remove an item from buffer to nextc  
    signal (mutex);  
    signal (empty);  
    // consume the item in nextc  
} while (TRUE);
```

- Một tập dữ liệu được chia sẻ đồng thời bởi một số tiến trình
 - Người đọc: chỉ đọc dữ liệu, không cập nhật hay thay đổi dữ liệu
 - Người viết: đọc và viết dữ liệu
- Cho phép nhiều người cùng đọc dữ liệu một lúc
 - Chỉ một người viết được thay đổi dữ liệu
- Dữ liệu chung
 - Tập dữ liệu
 - Khởi tạo semaphore **mutex** bằng 1 //Đảm bảo tính loại trừ lẫn nhau khi cập nhật **readcount**
 - Khởi tạo semaphore **wrt** bằng 1 //Đảm bảo tính loại trừ lẫn nhau của người viết
 - Khởi tạo semaphore biến tự nhiên **readcount** bằng 0 // Đếm số tiến trình đang đọc dữ liệu

Bài toán người đọc – Người viết

30

■ Tiến trình người viết

```
do {  
    wait (wrt) ;  
    //  writing is performed  
    signal (wrt) ;  
} while (TRUE);
```

Bài toán người đọc – Người viết

31

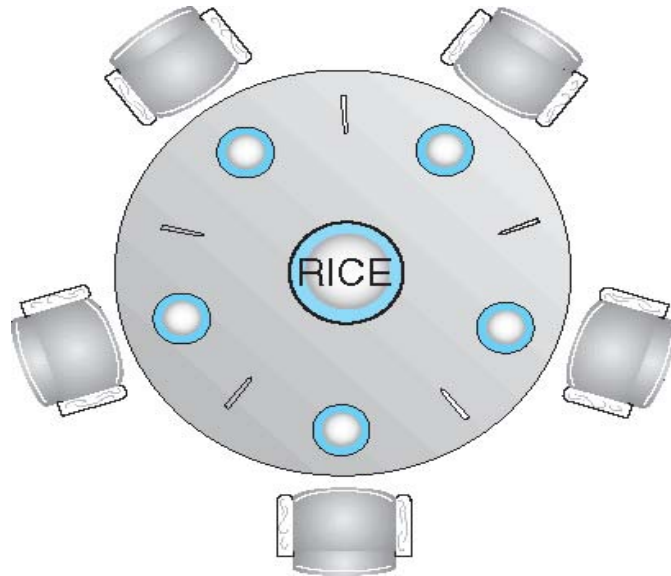
■ Tiến trình người đọc:

```
do {  
    wait (mutex) ;  
    readcount ++ ;  
    if (readcount == 1)  
        wait (wrt) ;  
    signal (mutex);  
  
    // reading is performed
```

```
wait (mutex) ;  
readcount - - ;  
if (readcount == 0)  
    signal (wrt) ;  
signal (mutex) ;  
} while (TRUE);
```

Bài toán bữa ăn nhà triết học

32



■ Dữ liệu chung:

- Bát cơm (Tập dữ liệu)
- Khởi tạo semaphore **đũa [5]** bằng 1

- Tiến trình cho nhà triết học i :

```
do {  
    wait (đũa[i] );  
    wait (đũa[ (i + 1) % 5] );  
    // eat  
    signal (đũa[i] );  
    signal (đũa[ (i + 1) % 5] );  
    // think  
} while (TRUE);
```

- Có thể xảy ra tình trạng bế tắc khi tất cả các nhà triết học đều đói và lấy đũa một lúc
- Khắc phục:
 - Cho phép nhiều nhất 4 nhà triết học cùng ngồi trên bàn ăn
 - Cho phép nhà triết học lấy đũa khi cả hai chiếc đũa ở trạng thái sẵn sàng
 - Nhà triết học có số hiệu lẻ lấy đũa bên trái, rồi bên phải. Ngược lại đối với các nhà triết học có số hiệu chẵn
- ***Yêu cầu:*** Chọn một giải pháp khắc phục tình trạng bế tắc, viết lại tiến trình cho nhà triết học i

Phần 5: Đồng bộ hoá tiến trình

5.6 Một số vấn đề với Semaphore

Một số vấn đề với Semaphore

35

- Sử dụng sai hàm:
 - `signal(mutex) .... wait(mutex)`
 - `wait(mutex) ... wait(mutex)`
 - Bỏ qua `wait(mutex)` hoặc `signal(mutex)` (hoặc cả hai)
- Bể tắc
- Nạn đói

Bế tắc

- **Bế tắc:** Hai tiến trình hoặc nhiều hơn đang đợi mãi mãi cho một sự kiện được tạo ra do một trong các tiến trình đang đợi này
- VD: Hai semaphore S và Q được khởi tạo bằng 1

P_0	P_1
wait (S);	wait (Q);
wait (Q);	wait (S);
.	.
.	.
.	.
signal (S);	signal (Q);
signal (Q);	signal (S);

Nạn đói

- **Nạn đói:** một tiến trình bị ngăn chặn mãi mãi trong một hàng đợi semaphore
- **Ưu tiên nghịch đảo:** Vấn đề lập lịch khi tiến trình có thứ tự ưu tiên thấp giữ khoá cần bởi tiến trình có thứ tự ưu tiên cao hơn
 - Vd: Ba tiến trình L, M, H có thứ tự ưu tiên $p_L < p_M < p_H$, tiến H cần tài nguyên R đang dùng bởi tiến trình L. Giả sử rằng tiến trình M chuyển sang trạng thái chạy và dùng tiến trình L.
 - **Giải pháp:** sử dụng phương thức **thừa kế thứ tự ưu tiên**. Tức là các tiến trình đang sử dụng nguồn tài nguyên cần bởi tiến trình có thứ tự ưu tiên cao sẽ tạm thời kế thừa thứ tự ưu tiên cao đến khi sử dụng xong.

Có một cái cầu bắc qua một dòng sông đầy cá sấu. Ở giữa cầu có một chông các thỏi vàng. Cách duy nhất để lấy được các thỏi vàng đó là đi qua một cái cổng, đi lên cầu, nhặt thỏi vàng và quay trở lại cổng. Ban đầu cổng mở, nếu một người bước lên cầu mà không có ai khác giữ cổng thì cổng sẽ bị đóng thì người đó sẽ bị chết đói chết cầu. Nếu có hai người ở trên cầu cùng một lúc thì cầu sẽ gãy và cá sấu sẽ không bị đói. Sau khi người cuối cùng lấy thỏi vàng cuối cùng thì những người còn lại không nên bước lên cầu. Sử dụng các hàm nguyên tử sau:

- *start_holding_gate ()* // bắt đầu giữ cổng
- *stop_holding_gate ()* // kết thúc giữ cổng
- *cross_bridge_and_get_gold ()* // lên cầu và lấy vàng, giá trị trả về là TRUE nếu còn vàng trên cầu

Viết hàm thực hiện nhiệm vụ lấy vàng của mỗi người sao cho: mỗi người đến đều lấy được một thỏi vàng cho đến khi hết vàng và không ai phải chờ đợi mãi mãi. Sử dụng semaphore để tránh hiện tượng tranh chấp

Đọc thêm

39

- Monitor
- Đồng bộ hoá trong Windows, Linux, Mac OS